

Lab Chương 2: Thực Hành Spring Core & Dependency Injection

Công cụ cần có: JDK 17+, Maven 3.8+, IntelliJ IDEA (hoặc VS Code)

Mục lục

- [Lab 1 — Tạo Bean và Tiêm Phụ Thuộc \(DI\)](#)
 - [Lab 2 — Debug Vòng Đời của Bean](#)
 - [Lab 3 — Cấu Hình Spring Boot Cơ Bản](#)
 - [Tự nghiên cứu thêm](#)
-

LAB 1 — Tạo Bean và Tiêm Phụ Thuộc (DI)

Mục tiêu

- Khai báo Spring Bean bằng @Component và @Bean
 - Thiết lập DI qua Constructor Injection
 - Hiểu cách DI hoạt động với Interface (abstraction)
 - Xử lý nhiều Bean cùng kiểu bằng @Qualifier
-

Cấu trúc project

```
lab1-spring-di/  
├── pom.xml  
└── src/main/java/com/lab/  
    ├── Main.java  
    ├── config/AppConfig.java  
    ├── model/Comment.java  
    ├── services/CommentService.java  
    ├── repositories/  
    │   ├── CommentRepository.java    <- interface  
    │   ├── DBCommentRepository.java  
    │   └── FileCommentRepository.java  
    └── proxies/  
        ├── CommentNotificationProxy.java <- interface  
        └── EmailNotificationProxy.java
```

Bước 0 — Tạo project Maven (3 phút)

pom.xml:

```
<project>  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.lab</groupId>  
  <artifactId>lab1-spring-di</artifactId>  
  <version>1.0</version>  
  
  <dependencies>  
    <dependency>
```

```
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>6.1.4</version>
</dependency>
</dependencies>
</project>
```

Bước 1 — Tạo Model và Interface (7 phút)

model/Comment.java:

```
package com.lab.model;

public class Comment {
    private String author;
    private String text;

    public Comment(String author, String text) {
        this.author = author;
        this.text = text;
    }

    public String getAuthor() { return author; }
    public String getText() { return text; }

    @Override
    public String toString() { return "[" + author + "]: " + text; }
}
```

repositories/CommentRepository.java (interface):

```
package com.lab.repositories;
import com.lab.model.Comment;

public interface CommentRepository {
    void storeComment(Comment comment);
}
```

proxies/CommentNotificationProxy.java (interface):

```
package com.lab.proxies;
import com.lab.model.Comment;

public interface CommentNotificationProxy {
    void sendNotification(Comment comment);
}
```

Bước 2 — Implement các Interface (5 phút)

repositories/DBCommentRepository.java:

```
package com.lab.repositories;

import com.lab.model.Comment;
import org.springframework.stereotype.Repository;

@Repository
public class DBCommentRepository implements CommentRepository {

    @Override
    public void storeComment(Comment comment) {
        System.out.println(" [DB] Storing: " + comment);
    }
}
```

proxies/EmailNotificationProxy.java:

```
package com.lab.proxies;

import com.lab.model.Comment;
import org.springframework.stereotype.Component;

@Component
public class EmailNotificationProxy implements CommentNotificationProxy {

    @Override
    public void sendNotification(Comment comment) {
        System.out.println(" [Email] Notification: " + comment);
    }
}
```

```
}  
}
```

Bước 3 — Tạo Service với Constructor Injection (8 phút)

services/CommentService.java:

```
package com.lab.services;  
  
import com.lab.model.Comment;  
import com.lab.proxies.CommentNotificationProxy;  
import com.lab.repositories.CommentRepository;  
import org.springframework.stereotype.Service;  
  
@Service  
public class CommentService {  
  
    // Constructor Injection: field final -> immutable -> thread-safe  
    private final CommentRepository commentRepository;  
    private final CommentNotificationProxy notificationProxy;  
  
    public CommentService(CommentRepository commentRepository,  
        CommentNotificationProxy notificationProxy) {  
        this.commentRepository = commentRepository;  
        this.notificationProxy = notificationProxy;  
        System.out.println("[INIT] CommentService created!");  
    }  
  
    public void publishComment(Comment comment) {  
        System.out.println("\n>> Publishing: " + comment);  
        commentRepository.storeComment(comment);  
        notificationProxy.sendNotification(comment);  
        System.out.println(">> Done!\n");  
    }  
}
```

Lưu ý: Khi class chỉ có 1 constructor, @Autowired là không bắt buộc. Spring tự động inject thông qua constructor duy nhất đó.

Bước 4 — Cấu hình và chạy (7 phút)

config/AppConfig.java:

```
package com.lab.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = {
    "com.lab.services",
    "com.lab.repositories",
    "com.lab.proxies"
})
public class AppConfig { }
```

Main.java:

```
package com.lab;

import com.lab.config.AppConfig;
import com.lab.model.Comment;
import com.lab.services.CommentService;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        System.out.println("=== Khoi tao Spring Context ===");
        var context = new AnnotationConfigApplicationContext(AppConfig.class);

        var service = context.getBean(CommentService.class);
        service.publishComment(new Comment("Alice", "Spring DI hoạt động!"));

        // Kiem tra Singleton: hai lan getBean() -> cung instance?
        var service2 = context.getBean(CommentService.class);
        System.out.println("Cung instance? " + (service == service2));
    }
}
```

```
context.close();  
}  
}
```

Kết quả mong đợi:

```
=== Khoi tao Spring Context ===  
[INIT] CommentService created!
```

```
>> Publishing: [Alice]: Spring DI hoạt động!  
[DB] Storing: [Alice]: Spring DI hoạt động!  
[Email] Notification: [Alice]: Spring DI hoạt động!  
>> Done!
```

```
Cung instance? true
```

Quan sát: [INIT] CommentService created! xuất hiện khi **khởi tạo Context**, không phải khi `getBean()` được gọi. Đây là Eager instantiation.

Bước 5 — Thử nghiệm A: `@Bean` thay cho `@Component` (10 phút)

Tình huống: `DBCommentRepository` đến từ thư viện ngoài — bạn không thể thêm `@Repository` vào.

1. Xóa `@Repository` khỏi `DBCommentRepository`
2. Cập nhật `AppConfig` để khai báo bean thủ công:

```
@Configuration  
@ComponentScan(basePackages = {"com.lab.services", "com.lab.proxies"})  
public class AppConfig {  
  
    @Bean  
    public CommentRepository commentRepository() {  
        return new DBCommentRepository(); // khai báo tuong minh  
    }  
}
```

3. Chạy lại — kết quả **không thay đổi**.

Kết luận: @Bean và @Component đều tạo ra Bean trong Context. Dùng @Bean khi class đến từ thư viện (không sửa được source code). Dùng @Component khi là class của chính mình.

Bước 6 — Thử nghiệm B: Nhiều Bean cùng kiểu — @Qualifier (10 phút)

Thêm một implementation thứ hai:

repositories/FileCommentRepository.java:

```
package com.lab.repositories;

import com.lab.model.Comment;
import org.springframework.stereotype.Repository;

@Repository
public class FileCommentRepository implements CommentRepository {

    @Override
    public void storeComment(Comment comment) {
        System.out.println(" [File] Storing: " + comment);
    }
}
```

Khôi phục @Repository trên DBCommentRepository (nếu đã xóa ở bước trên).

Chạy lại → Spring báo lỗi:

```
NoUniqueBeanDefinitionException: expected single matching bean
but found 2: DBCommentRepository, FileCommentRepository
```

Sửa bằng @Qualifier trong CommentService:

```
public CommentService(
    @Qualifier("DBCommentRepository") CommentRepository repo,
    CommentNotificationProxy proxy) {
    this.commentRepository = repo;
    this.notificationProxy = proxy;
    System.out.println("[INIT] CommentService created!");
}
```


Thử thêm: Đổi `@Qualifier("fileCommentRepository")` → quan sát output thay đổi. Tên bean mặc định = tên class viết thường chữ cái đầu.

Câu hỏi ôn tập Lab 1 (5 phút)

1. Sự khác biệt giữa `@Bean` và `@Component`? Khi nào dùng cái nào?
 2. Tại sao Constructor Injection được ưa hơn Field Injection (`@Autowired` trên field)?
 3. `@Qualifier` giải quyết vấn đề gì? Thay thế nó bằng `@Primary` — hành vi thay đổi thế nào?
 4. Comment (POJO) có nên là Bean không? Vì sao?
-

LAB 2 — Debug Vòng Đời của Bean

Mục tiêu

- Quan sát thứ tự khởi tạo Bean theo dependency graph
 - Dùng `@PostConstruct` / `@PreDestroy` để hook vào vòng đời Bean
 - So sánh hành vi Singleton và Prototype qua log thực tế
-

Cấu trúc project

```
lab2-bean-lifecycle/  
├── pom.xml  
└── src/main/java/com/lab/lifecycle/  
    ├── Main.java  
    ├── config/AppConfig.java  
    ├── singleton/  
    │   ├── ServiceA.java  
    │   └── ServiceB.java  
    ├── prototype/  
    │   └── RequestProcessor.java  
    └── managed/  
        └── ManagedBean.java
```

Thêm dependency vào pom.xml:

```
<!-- Spring Context -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.4</version>
</dependency>
<!-- @PostConstruct / @PreDestroy -->
<dependency>
  <groupId>jakarta.annotation</groupId>
  <artifactId>jakarta.annotation-api</artifactId>
  <version>2.1.1</version>
</dependency>
```

Bước 1 — Thứ tự khởi tạo Bean (10 phút)

singleton/ServiceB.java:

```
package com.lab.lifecycle.singleton;
import org.springframework.stereotype.Service;

@Service
public class ServiceB {

    public ServiceB() {
        System.out.println("[INIT] ServiceB created");
    }

    public void assist() {
        System.out.println(" ServiceB.assist() called");
    }
}
```

singleton/ServiceA.java:

```
package com.lab.lifecycle.singleton;
import org.springframework.stereotype.Service;

@Service
```

```

public class ServiceA {

    private final ServiceB serviceB;

    public ServiceA(ServiceB serviceB) {
        this.serviceB = serviceB;
        System.out.println("[INIT] ServiceA created (ServiceB injected)");
    }

    public void doWork() {
        System.out.println("ServiceA.doWork() ->");
        serviceB.assist();
    }
}

```

config/AppConfig.java:

```

@Configuration
@ComponentScan(basePackages = "com.lab.lifecycle")
public class AppConfig { }

```

Main.java (phần 1):

```

System.out.println("--- Before context ---");
var ctx = new AnnotationConfigApplicationContext(AppConfig.class);
System.out.println("--- After context ---\n");

ctx.getBean(ServiceA.class).doWork();
ctx.close();

```

Kết quả mong đợi:

```

--- Before context ---
[INIT] ServiceB created      <- ServiceB trước (vì ServiceA cần nó)
[INIT] ServiceA created (ServiceB injected)
--- After context ---

ServiceA.doWork() ->
ServiceB.assist() called

```

Quan sát quan trọng: Bean được tạo **trước khi** --- After context --- xuất hiện. Spring tính toán thứ tự dựa trên dependency graph: ServiceA phụ thuộc ServiceB → ServiceB phải tồn tại trước.

Bước 2 — @PostConstruct & @PreDestroy (15 phút)

managed/ManagedBean.java:

```
package com.lab.lifecycle.managed;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Component;

@Component
public class ManagedBean {

    public ManagedBean() {
        System.out.println(" [1] Constructor: object duoc tao");
    }

    @PostConstruct
    public void init() {
        // Chay SAU constructor + DI hoan tat
        // Dung de: khoi tao cache, ket noi DB, validate config...
        System.out.println(" [2] @PostConstruct: bean san sang");
    }

    public void doWork() {
        System.out.println(" [*] doWork() duoc goi");
    }

    @PreDestroy
    public void cleanup() {
        // Chay TRUOC khi bean bi huy (context.close())
        // Dung de: dong connection, flush buffer, giai phong resource...
        System.out.println(" [3] @PreDestroy: don dep truoc khi huy");
    }
}
```

Main.java (phần 2):

```
System.out.println("\n=== Vòng doi Bean ===");
var ctx = new AnnotationConfigApplicationContext(AppConfig.class);

var bean = ctx.getBean(ManagedBean.class);
bean.doWork();

System.out.println("\n--- Dong context (ctx.close()) ---");
ctx.close();
```

Kết quả mong đợi:

```
=== Vòng doi Bean ===
[1] Constructor: object duoc tao
[2] @PostConstruct: bean san sang
[*] doWork() duoc goi

--- Dong context (ctx.close()) ---
[3] @PreDestroy: don dep truooc khi huy
```

Sơ đồ vòng đời đầy đủ:

```
[1] Spring gọi Constructor
    |
[2] Spring inject Dependency (DI)
    |
[3] @PostConstruct  <- bean san sang su dung
    |
    [Bean hoạt động]
    |
[4] context.close()
    |
[5] @PreDestroy    <- giai phong resource
    |
    Bean bị huy
```

Thử nghiệm thêm: Thêm @Autowired field vào ManagedBean, in giá trị trong @PostConstruct — đảm bảo DI đã xảy ra trước khi init() chạy.

Bước 3 — Singleton vs. Prototype (15 phút)

prototype/RequestProcessor.java:

```
package com.lab.lifecycle.prototype;

import org.springframework.beans.factory.config.BeanDefinition;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope(BeanDefinition.SCOPE_PROTOTYPE)
public class RequestProcessor {

    private static int counter = 0;
    private final int id;

    public RequestProcessor() {
        this.id = ++counter;
        System.out.println(" [NEW] RequestProcessor #" + id + " created");
    }

    public void process(String data) {
        System.out.println(" Processor #" + id + " -> " + data);
    }
}
```

Main.java (phần 3 — chạy sau phần 2):

```
System.out.println("\n=== Singleton ===");
var a1 = ctx.getBean(ServiceA.class);
var a2 = ctx.getBean(ServiceA.class);
System.out.println("a1 == a2 ? " + (a1 == a2));    // -> true

System.out.println("\n=== Prototype ===");
var p1 = ctx.getBean(RequestProcessor.class);
var p2 = ctx.getBean(RequestProcessor.class);
var p3 = ctx.getBean(RequestProcessor.class);

p1.process("Request A");
```

```

p2.process("Request B");
p3.process("Request C");

System.out.println("p1 == p2 ? " + (p1 == p2));    // -> false
System.out.println("p1 == p3 ? " + (p1 == p3));    // -> false

```

Kết quả mong đợi:

```

=== Singleton ===
a1 == a2 ? true          <- cung 1 instance

=== Prototype ===
[NEW] RequestProcessor #1 created
[NEW] RequestProcessor #2 created
[NEW] RequestProcessor #3 created
Processor #1 -> Request A
Processor #2 -> Request B
Processor #3 -> Request C
p1 == p2 ? false        <- khác instance
p1 == p3 ? false

```

Bẫy thường gặp — inject Prototype vào Singleton:

```

@Service
public class SomeService {
    @Autowired
    private RequestProcessor processor; // BAD: chỉ inject 1 lần duy nhất!
}

```

SomeService là singleton → Spring inject RequestProcessor **1 lần** khi tạo singleton. Dù RequestProcessor là prototype, bạn luôn nhận được **cùng 1 instance**.

Sửa đúng: inject ApplicationContext và gọi getBean() bên trong method.

Câu hỏi ôn tập Lab 2 (không tính giờ — làm sau)

1. Tại sao ServiceB được khởi tạo trước ServiceA?
2. @PostConstruct khác gì so với việc viết code khởi tạo trong Constructor?
3. Singleton bean có attribute mutable (có thể thay đổi) gây ra vấn đề gì trong môi trường multi-thread?

4. Tại sao inject Prototype vào Singleton bằng @Autowired thông thường lại sai?
-
-

LAB 3 — Cấu Hình Spring Boot Cơ Bản

Mục tiêu

- Khởi tạo project Spring Boot và hiểu @SpringBootApplication
 - Ngoại hoá cấu hình bằng application.properties
 - Inject config vào Bean với @Value
 - Chạy code khởi động bằng CommandLineRunner
 - Chuyển đổi môi trường bằng Spring Profiles
-
-

Cấu trúc project

```
lab3-spring-boot/  
├── pom.xml  
└── src/main/  
    ├── java/com/lab/boot/  
    │   ├── Lab3Application.java  
    │   ├── services/NotificationService.java  
    │   └── runner/AppRunner.java  
    └── resources/  
        ├── application.properties  
        ├── application-dev.properties  
        └── application-prod.properties
```

Bước 0 — Tạo Spring Boot Project (3 phút)

pom.xml:

```
<project>  
  <modelVersion>4.0.0</modelVersion>  
  <parent>  
    <groupId>org.springframework.boot</groupId>
```



```

    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.3</version>
  </parent>
  <groupId>com.lab</groupId>
  <artifactId>lab3-spring-boot</artifactId>
  <version>1.0</version>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter</artifactId>
    </dependency>
  </dependencies>
</project>

```

Mẹo nhanh: Tạo project tự động tại start.spring.io — chọn **Spring Boot 3.x**, dependency **Spring Web**, download và mở trong IDE.

Bước 1 — @SpringBootApplication là gì? (5 phút)

Lab3Application.java:

```

package com.lab.boot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
// Tương đương viết tay 3 annotation:
// @Configuration      <- đây là configuration class
// @ComponentScan      <- quét tất cả package con tu đây
// @EnableAutoConfiguration <- Spring Boot tu cấu hình theo classpath
public class Lab3Application {
    public static void main(String[] args) {
        SpringApplication.run(Lab3Application.class, args);
    }
}

```

So sánh Lab 1 vs Lab 3:

	Lab 1 (Spring Core)	Lab 3 (Spring Boot)
Khởi tạo	new AnnotationConfigApplicati onContext(AppConfig.class)	SpringApplication.run(...)
Config class	Phải tự viết AppConfig.java	Auto-configuration
@ComponentScan	Phải khai báo tường minh	Tự động từ package gốc

Bước 2 — application.properties và @Value (12 phút)

resources/application.properties:

```
spring.application.name=Lab3-SpringBoot
```

```
app.notification.channel=email
app.notification.from=no-reply@lab.com
app.notification.max-retries=3
app.notification.enabled=true
```

services/NotificationService.java:

```
package com.lab.boot.services;
```

```
import org.springframework.beans.factory.annotation.Value;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class NotificationService {
```

```
    @Value("${app.notification.channel}")
```

```
    private String channel;
```

```
    @Value("${app.notification.from}")
```

```
    private String fromAddress;
```

```
    @Value("${app.notification.max-retries:3}") // gia tri mac dinh neu key khong ton tai
```

```
    private int maxRetries;
```

```

@Value("${app.notification.enabled:true}")
private boolean enabled;

public void send(String message) {
    if (!enabled) {
        System.out.println("[WARN] Notification disabled.");
        return;
    }
    System.out.printf("[%s] From: %s | %s (retry=%d)%n",
        channel.toUpperCase(), fromAddress, message, maxRetries);
}
}

```

Bước 3 — CommandLineRunner (5 phút)

CommandLineRunner cho phép chạy code ngay sau khi Spring Context sẵn sàng.

runner/AppRunner.java:

```

package com.lab.boot.runner;

import com.lab.boot.services.NotificationService;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class AppRunner implements CommandLineRunner {

    private final NotificationService notificationService;

    public AppRunner(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    @Override
    public void run(String... args) {
        System.out.println("\n=== App ready — running tasks ===");
        notificationService.send("He thong da khoi dong thanh cong!");
    }
}

```

```

        notificationService.send("Tat ca service san sang.");
        System.out.println("=====\\n");
    }
}

```

Kết quả mong đợi:

```

=== App ready — running tasks ===
[EMAIL] From: no-reply@lab.com | He thong da khoi dong thanh cong! (retry=3)
[EMAIL] From: no-reply@lab.com | Tat ca service san sang. (retry=3)
=====

```

Bước 4 — Spring Profiles (5 phút)

resources/application-dev.properties:

```

app.notification.channel=console
app.notification.from=dev@localhost
app.notification.enabled=true

```

resources/application-prod.properties:

```

app.notification.channel=email
app.notification.from=system@company.com
app.notification.enabled=true
app.notification.max-retries=5

```

Kích hoạt profile trong application.properties:

```

spring.profiles.active=dev

```

Hoặc qua command line (không cần sửa file):

```

java -jar lab3.jar --spring.profiles.active=prod

```

Thử nghiệm: 1. Chạy với spring.profiles.active=dev → channel=console, from=dev@localhost 2. Đổi sang prod → channel=email, from=system@company.com

Cơ chế: Spring Boot load application.properties trước, sau đó application-{profile}.properties ghi đè các key trùng nhau.

Câu hỏi ôn tập Lab 3 (không tính giờ — làm sau)

1. @SpringBootApplication gồm những annotation nào? Mỗi cái làm gì?
 2. Sự khác biệt giữa @Value("\${key}") và @Value("\${key:default}")?
 3. CommandLineRunner.run() được gọi khi nào trong vòng đời ứng dụng?
 4. Khi đặt spring.profiles.active=prod, application.properties có còn được đọc không?
-
-

Tự nghiên cứu thêm (ngoài giờ lab)

Các nội dung dưới đây **không bắt buộc trong 120 phút**. Học viên có thể tự thực hành sau giờ học.

A. Field Injection và nhược điểm

```
@Service
public class CommentService {
    @Autowired                // Field Injection
    private CommentRepository repo; // không the final

    @Autowired
    private CommentNotificationProxy proxy;
}
```

Nhược điểm: - Field không thể final → không immutable → dễ race condition trong môi trường multi-thread - Khó viết unit test: phải dùng reflection để mock dependency

Bài tập: Viết unit test cho cả 2 phiên bản (Field Injection vs Constructor Injection). Thấy ngay cái nào dễ test hơn.

B. registerBean() — Thêm Bean theo điều kiện

```
var context = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
boolean useFile = "file".equals(System.getenv("REPO_TYPE"));
if (useFile) {
    context.registerBean("commentRepository",
        FileCommentRepository.class,
        FileCommentRepository::new);
}
```

```
var service = context.getBean(CommentService.class);
```

Khi nào dùng: Cần quyết định runtime (đọc biến môi trường, file config, feature flag...) mà @Bean và @Component không đủ linh hoạt.

C. Eager vs. Lazy Instantiation

Thêm @Lazy vào bean tốn tài nguyên:

```
@Service
@Lazy           // chỉ tạo khi có ai getBean() lan đầu
public class HeavyService {
    public HeavyService() {
        System.out.println("[INIT] HeavyService — expensive!");
    }
}
```

Quan sát: không có @Lazy → [INIT] xuất hiện khi context khởi tạo. Có @Lazy → [INIT] chỉ xuất hiện khi bean được yêu cầu lần đầu.

Khi nào dùng @Lazy: Bean tốn tài nguyên khởi tạo và không phải lúc nào cũng cần.

D. Circular Dependency — Phát hiện và sửa

Cố tình tạo circular dependency:

```
@Service
public class ChickenService {
    public ChickenService(EggService e) {}
}
```

```
@Service
public class EggService {
    public EggService(ChickenService c) {}
}
```

Lỗi:

BeanCurrentlyInCreationException:
Is there an unresolvable circular reference?

Cách sửa: Tách phần shared thành bean thứ ba, phá vỡ vòng tròn:

```
@Service public class LifecycleService { ... }
```

```
@Service public class ChickenService {
    ChickenService(LifecycleService s) { ... }
}
```

```
@Service public class EggService {
    EggService(LifecycleService s) { ... }
}
```

E. @ConfigurationProperties — Nhóm config theo prefix

Thay vì nhiều @Value riêng lẻ, dùng một class để nhóm:

```
@Component
@ConfigurationProperties(prefix = "app.notification")
public class NotificationConfig {
    private String channel;
    private String from;
    private int    maxRetries = 3;
    private boolean enabled   = true;

    // PHAI co getters + setters de Spring bind
    public String getChannel() { return channel; }
    public void setChannel(String v) { this.channel = v; }
    // ...
}
```

Inject vào Service qua constructor:

```
@Service
public class NotificationService {
    private final NotificationConfig config;
    public NotificationService(NotificationConfig config) {
        this.config = config;
    }
}
```

Ưu điểm: Type-safe, tái sử dụng, dễ validate với @Validated.

F. Unit Test với @SpringBootTest

```
@SpringBootTest
@ActiveProfiles("dev")
class NotificationServiceTest {

    @Autowired NotificationService service;

    @Test
    void contextLoads() {
        assertThat(service).isNotNull();
    }

    @Test
    void sendDoesNotThrow() {
        service.send("test message");
    }
}
```
